Algoritmos de clusterización

M7: Aprendizaje de máquina no supervisado

|**AE4**: Implementar un modelo predictivo aplicando algoritmos de clusterización utilizando lenguaje python para resolver un problema.

alkemy

Introducción 👩‍💻

El aprendizaje no supervisado es una disciplina clave en el campo del Machine Learning, y dentro de ella, la clusterización ocupa un lugar fundamental. Esta técnica permite agrupar datos sin etiquetas previas, basándose únicamente en similitudes o distancias entre observaciones. Su utilidad se extiende a distintos campos como la biología, el marketing, la astronomía y la informática.

Los algoritmos de clusterización ayudan a descubrir patrones ocultos, a clasificar automáticamente elementos y a reducir la complejidad de los datos. En esta lección nos enfocaremos en los principales algoritmos de agrupamiento disponibles, su implementación en Python, y su aplicabilidad a problemas reales.

Analizaremos con profundidad K-Means, agrupamiento jerárquico y DBSCAN. Cada uno con sus particularidades, ventajas y desventajas. También aprenderás cómo elegir el número óptimo de clusters y evaluar los resultados obtenidos a través de visualizaciones y métricas.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Diferenciar entre los principales algoritmos de clusterización y sus características.
- Determinar cuál es el algoritmo más adecuado según el tipo de datos y objetivo analítico.
- Implementar modelos de clusterización en Python desde cero.
- Interpretar visualizaciones como dendrogramas y gráficas de silueta para evaluar resultados.

¿Qué es la clusterización?

La clusterización, o agrupamiento, es una técnica de análisis exploratorio que permite identificar grupos o segmentos en un conjunto de datos sin necesidad de conocer etiquetas previas. Se basa en la similitud de características y busca agrupar observaciones que se parezcan entre sí y estén lo más alejadas posible de otros grupos.

Esta técnica es utilizada en una amplia gama de aplicaciones como segmentación de clientes, agrupación de documentos, detección de anomalías, reducción de ruido y compresión de datos. Su flexibilidad la hace ideal para descubrir patrones cuando no hay una salida definida.

El resultado de una clusterización no supervisada puede usarse para comprender mejor la estructura de los datos, hacer recomendaciones personalizadas o generar etiquetas automáticas que luego alimenten algoritmos supervisados.

Una de las dificultades comunes al aplicar clusterización es definir el número óptimo de grupos (clusters) y seleccionar el algoritmo adecuado según la forma, densidad y ruido en los datos.

Para evaluar el resultado de un modelo de agrupamiento, se suelen utilizar medidas como el coeficiente de silueta, la cohesión intra-cluster y la separación inter-cluster.

Agrupamiento Jerárquico

El agrupamiento jerárquico crea una estructura en forma de árbol que representa las relaciones entre observaciones. Este árbol se conoce como dendrograma. Existen dos tipos principales de agrupamiento jerárquico: aglomerativo y divisivo.

- **Aglomerativo**: parte con cada punto como un clúster individual y va fusionando los más cercanos hasta agruparlos en uno solo.
- **Divisivo**: inicia con todos los puntos en un solo grupo y los va dividiendo progresivamente.

El dendrograma ayuda a decidir visualmente el número óptimo de clusters. Para aplicar este tipo de agrupamiento se siguen varias etapas:

1. Normalización de datos.
2. Cálculo de la matriz de distancias.

3. Aplicación del algoritmo de fusión (por ejemplo: enlace simple, completo o promedio).
4. Visualización del dendrograma.

Ventajas:

- No requiere definir el número de clusters.
- Proporciona una vista jerárquica clara.

Desventajas:

- Escalabilidad limitada en grandes conjuntos.
- Alta sensibilidad al ruido y outliers.

Implementación en Python:

```
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
import pandas as pd

# Datos de ejemplo
X = pd.read_csv("datos.csv")
Z = linkage(X, 'ward')

plt.figure(figsize=(10, 7))
dendrogram(Z)
plt.title("Dendrograma")
plt.xlabel("Observaciones")
plt.ylabel("Distancia")
plt.show()
```

Algoritmo K-Means

K-Means es un algoritmo que agrupa datos en K clústeres definidos por sus centroides. El proceso es iterativo y busca minimizar la varianza dentro de los grupos.

Etapas del algoritmo:

1. Elegir K (cantidad de grupos).
2. Inicializar los centroides aleatoriamente.
3. Asignar cada punto al centroide más cercano.
4. Recalcular centroides.
5. Repetir hasta que no haya cambios significativos.

Selección del valor de K:

- *Método del codo*:

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []

for k in range(1, 10):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)

plt.plot(range(1, 10), inertia, marker='o')
plt.xlabel('Número de Clusters')
plt.ylabel('Inercia')
plt.title('Método del Codo')
plt.show()
```

Evaluación del modelo:

- Coeficiente de silueta:

```
from sklearn.metrics import silhouette_score

score = silhouette_score(X, kmeans.labels_)
print("Coeficiente de Silueta:", score)
```

Ventajas:

- Simple y eficiente.
- Escalable a grandes volúmenes de datos.

Desventajas:

- Requiere definir K previamente.
- No funciona bien con formas no esféricas o con outliers.

Algoritmo DBSCAN

DBSCAN se basa en la densidad para formar clusters. Es útil para encontrar estructuras arbitrarias y detectar outliers.

Parámetros clave:

- `eps`: radio de vecindad.
- `min_samples`: puntos mínimos para considerar un clúster.

Funcionamiento:

- Clasifica los puntos como núcleo, frontera o ruido.
- Expande los clusters desde los puntos núcleo.

Implementación en Python:

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

db = DBSCAN(eps=0.5, min_samples=5)
db.fit(X_scaled)

labels = db.labels_
print(set(labels))  # -1 representa ruido
```

Ventajas:

- No necesita número de clusters.
- Identifica clusters de forma arbitraria.
- Resistente a outliers.

Desventajas:

- Sensible a la elección de parámetros.
- Disminuye su rendimiento en alta dimensión.

Cierre ✍️

El uso de algoritmos de clusterización en Python permite implementar soluciones prácticas a problemas del mundo real, desde la segmentación de clientes hasta la detección de patrones en datos masivos.

Comprender las diferencias entre agrupamiento jerárquico, K-Means y DBSCAN permite seleccionar el método más adecuado según la naturaleza del conjunto de datos y los objetivos del análisis.

La implementación de estas técnicas no sólo requiere conocer la teoría, sino también practicar con código real, evaluar métricas, ajustar parámetros y visualizar los resultados.

Python y sus bibliotecas (`scikit-learn`, `matplotlib`, `scipy`) proporcionan un entorno robusto para experimentar, analizar y validar modelos de clusterización.

Dominar estas técnicas fortalece habilidades clave para el análisis exploratorio de datos, resolución de problemas no supervisados y desarrollo de soluciones predictivas sin necesidad de etiquetas.

¡Nos vemos de nuevo, más adelante! 👋

Referencias📚

- Scikit-learn. (s.f.). *Clustering*. Recuperado el 28 de julio de 2025, de https://scikit-learn.org/stable/modules/clustering.html
- IBM. (2020). *K-means clustering*. Recuperado el 28 de julio de 2025, de https://www.ibm.com/cloud/learn/k-means
- Towards Data Science. (2021). *Understanding DBSCAN*. Recuperado el 28 de julio de 2025, de https://towardsdatascience.com/dbscan-clustering-explained-97556a2ad556
- GeeksforGeeks. (2021). *Hierarchical clustering*. Recuperado el 28 de julio de 2025, de https://www.geeksforgeeks.org/hierarchical-clustering/
- Analytics Vidhya. (2022). *Introduction to K-means clustering*. Recuperado el 28 de julio de 2025, de https://www.analyticsvidhya.com/blog/2019/08/comprehensive-guide-k-means-clustering/
- Machine Learning Mastery. (2022). *Clustering algorithms with Python*. Recuperado el 28 de julio de 2025, de https://machinelearningmastery.com/clustering-algorithms-with-python/

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy